
GameGen-Verifier: Parallel Keypoint-Based Verification for LLM-Generated Games via Runtime State Injection

Chaobo Jia^{1†}, Ruipeng Wan^{2†}, Ting Sun^{3†}, Weihao Tan⁴
 Borui Wan⁵, Yuxuan Tong⁶, Guangming Sheng⁵, Hong Xu¹
¹CUHK ²HUST ³Lionrock AI Lab ⁴NTU ⁵HKU ⁶Independent Researcher
[†]Equal contribution

Abstract

LLM-based game generation promises to turn natural-language specifications into executable games, but progress is limited by the lack of reliable automated verification. Unlike conventional code generation, game correctness is defined over long-horizon interaction: a game may appear correct while violating core mechanics such as state updates, interaction rules, and phase transitions. Existing Agent-as-a-Verifier approaches collapse verification into open-ended gameplay, making verdicts reachability-bound, time-consuming, coverage-limited, and sensitive to the agent’s gameplay ability.

We present GameGen-Verifier, an automated verification paradigm for LLM-generated games that decomposes a specification into *verifiable keypoints* and grounds them into independent verification units. Each unit patches the game runtime into a concrete target state, executes a bounded interaction, and judges the outcome against the keypoint assertion. We implement GGV-HARNESS, a scalable agentic harness providing concurrency management, runtime isolation, and fault recovery.

On VERIGAME, our dataset of 100 games across seven genres, GameGen-Verifier achieves up to 92.2% accuracy against human judgments versus 58.8% for the coverage-enforced Agent-as-a-Verifier baseline, while reducing wall-clock time by up to 16.6 $\times$.

1 Introduction

Historically, turning a game proposal into a playable artifact required expert development teams, specialized tooling, and long production cycles. Recent work has begun to explore LLM-based game generation systems that aim to rapidly turn natural-language specifications into executable, playable games [1, 3, 15, 20, 22]. However, progress in game generation is increasingly bottlenecked by the lack of reliable *verification* of whether generated games correctly implement their specifications [1, 22]. For conventional code generation, reliable tests enable benchmark construction, scalable data curation, and accurate reward signals that sustain the training flywheel [7, 18, 24, 27, 28]. For game generation, the analogous judge remains missing: a generated game may build successfully and appear visually correct while violating core mechanics such as state updates, interaction rules, and phase transitions. Without a verifier that agrees with human judgment on runtime logical correctness, benchmark construction remains ill-defined, high-quality training data cannot be scaled, and training lacks trustworthy feedback. Scalable verification is therefore a prerequisite for turning game generation from qualitative demonstrations into a well-defined research problem, as illustrated in Figure 1. The closest existing paradigm is Agent-as-a-Verifier (AaaV), in which an autonomous agent interacts with a generated artifact at runtime and verifies the observed behavior against the specification [25, 29–31, 43]. However, when applied directly to games, AaaV collapses verification

Preprint.

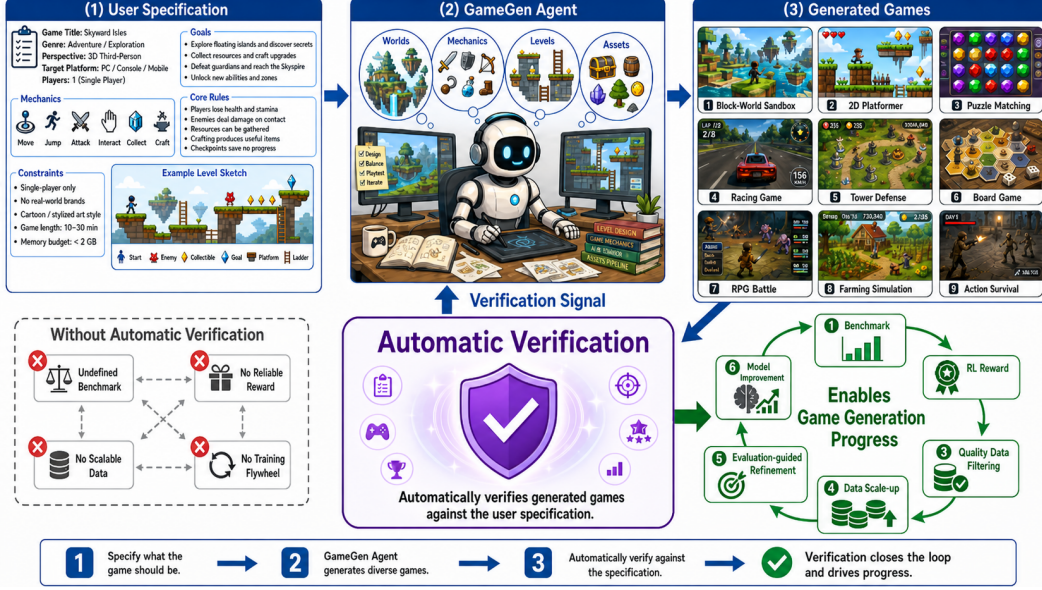


Figure 1: A user specification (1) drives a game generation agent that synthesizes worlds, mechanics, levels, and assets to produce playable games across diverse genres (2, 3). An automatic verifier then checks each generated game against the original specification and feeds correctness signals back to the agent, closing the generation loop and enabling downstream benchmarking, data curation, and model improvement (bottom right).

into open-ended gameplay, with the limitations summarized in Figure 2: gameplay rewards progress and exploration, whereas verification requires systematic coverage of specified mechanics and causal attribution of failures. Before checking any target mechanic, the verifier must also reach a state where it is exercised, making evaluation time-consuming, coverage-limited, and dependent on agents that are themselves unreliable at complex gameplay. Consequently, AaaV cannot serve as a scalable verifier for game generation.

To address this verification bottleneck, we make two key observations. First, many specification-level failures in generated games can be exposed by a sparse set of localized critical behaviors, including build and launch health, collision handling, reward settlement, state updates, rule triggers, phase transitions, and progression changes, rather than by exploring entire gameplay trajectories. Second, the states needed to exercise these behaviors need not be reached through gameplay: games are state machines whose runtime states are characterized by parameters, and for LLM-generated games, the verifier can identify this parameter structure from the white-box implementation and instantiate relevant states directly through runtime state-patching (Section 4.2) [9, 14, 16, 38].

Building on these observations, we present **GameGen-Verifier**, an automated verification paradigm for LLM-generated games. At the core of GameGen-Verifier is a keypoint abstraction: GameGen-Verifier extracts sparse, specification-derived critical conditions from the specification and formulates them as *verifiable keypoints*. Each keypoint is a localized behavioral assertion, casting correctness as a local, bounded check rather than a global trajectory-level judgment. To make these keypoints executable, GameGen-Verifier grounds each one into a state-grounded verification unit: the verifier constructs a precise game state described by the implementation’s parameter structure, injects it into the runtime, executes a bounded interaction sequence, and judges whether the observed outcome satisfies the expected one. This formulation also makes verification units self-contained, reducing unreliable gameplay to a finite set of parallelizable short-horizon verifications.

GameGen-Verifier closes the loop by attributing keypoint verdicts back to specification elements and propagating FAIL verdicts through their dependency structure.

We further propose GGV-HARNESS, a scalable agentic verification harness built on a two-layer orchestration-worker architecture that enforces concurrency management, runtime isolation, and fault recovery.

This paper makes the following contributions:



Figure 2: Applying AaaV to game verification forces the agent through long gameplay sequences to reach target mechanics, but the exponentially growing state space makes full coverage intractable and open-ended play provides no clear correctness oracle, making AaaV slow, coverage-limited, and misaligned with specification-level judgment.

- We identify the verification bottleneck in game generation and show why existing paradigms are insufficient for scalable correctness judgments.
- We propose GameGen-Verifier, a new verification paradigm that reformulates game verification from open-ended trajectory exploration into state-grounded verification of specification-derived verifiable keypoints.
- We implement a scalable agentic verification harness centered on three systems guarantees: concurrency management, runtime isolation, and fault recovery, enabling reliable large-scale verification execution.
- We validate GameGen-Verifier on VERIGAME and show that it achieves up to 92.2% Acc@5 and 95.4% F1@5 against human judgment, while reducing wall-clock time by up to $16.6\times$ compared with the coverage-enforced Agent-as-a-Verifier baseline.

2 Related Work

Evaluation paradigms and benchmarks for code generation. Evaluation of code generation has evolved alongside task complexity. Early benchmarks relied on deterministic test-based verification, where correctness is decided by predefined input-output test cases [6, 7, 18, 23]. As tasks expanded to full-stack systems, static tests became insufficient, leading to LLM-as-a-Judge approaches that score generated artifacts against human-defined rubrics [26, 30, 45]. More recently, Agent-as-a-Verifier (AaaV) approaches have emerged for interactive systems such as web applications, where autonomous agents interact with the live system to verify functionality [25, 29–31, 43]. AaaV is the closest existing evaluation paradigm to our setting because it verifies runtime behavior rather than static code alone. However, existing AaaV benchmarks are primarily designed for web applications, where states are largely discrete, interactions are short, and verification targets are typically reachable through a small number of UI actions.

Game testing. Game testing has a long tradition in both industrial practice and software-engineering research. Surveys of video-game testing report that production pipelines still rely heavily on manual playtesting and testers’ domain knowledge, while automated techniques remain difficult to generalize across genres and development workflows [34]. Representative approaches include runtime or model-based monitoring, which instruments the game loop or an abstract model to check temporal gameplay properties and expose bugs [41], and agent-based automated playtesting, where reinforcement-learning or MCTS-style agents generate play traces to search for defects or coverage gaps [5, 35]. In the context of LLM-generated games, Godogen [1] incorporates game-specific tests such as task-specific harnesses, execution-time assertions, and screenshot-based visual QA during the development process, while OpenGame [22] evaluates agentic web-game generation through build, visual usability, and intent-alignment signals. These techniques are useful for validating existing games or local

development-time properties, but they typically assume a human-authored target game, a handcrafted model or oracle, or a tester agent that still has to traverse gameplay states. They do not directly address scalable specification-level fidelity checking for games generated from natural language.

Coding agents. LLM-powered coding agents [2, 32, 33] have moved programming from one-shot synthesis to iterative agent-driven development within real execution environments, supporting workflows in which models inspect repositories, edit files, execute commands, and refine outputs based on environmental feedback. Multi-agent and agent-swarm systems further decompose tasks into sub-problems for coordinated parallel execution [21, 36, 42, 44]. Our agentic verification workflow draws on this paradigm but is scenario-specific and vendor-independent, designed for stable parallel evaluation rather than general-purpose task orchestration.

3 Background

LLM-based Game Generation. LLM-based game generation aims to turn a natural-language game proposal into an executable, playable artifact [1, 3, 15, 20, 22]. In our setting, the input is a *specification*: a natural-language document that describes the intended game, covering its objective, entities, player controls, rules, scoring or reward logic, failure conditions, progression structure, and other aspects. The generator is then asked to realize this specification as a complete interactive system, synthesizing code, assets, and supporting structures for runtime loops, entity and state management, gameplay logic, input handling, collision or physics behavior, UI, and other game-specific components.

Game-generation Stack. Recent LLM-based game generation systems largely target web-stack games built in JavaScript, TypeScript, and HTML [1, 22]. Mainstream engines such as Godot, Unity, and Unreal Engine are beginning to receive community MCP (Model Context Protocol) integrations that expose scene and object inspection, node or object creation, and script editing to agents [8, 10, 11], but this support remains fragmented and immature, making these engines difficult to use as routine generation stacks.

Hoare triples. Hoare logic is a classical formalism for reasoning about program correctness through triples of the form $\{P\} C \{Q\}$ [19]. Here, P is a precondition over the program state, C is a command or program fragment to be executed, and Q is a postcondition over the resulting state. Under the standard partial-correctness interpretation, the triple asserts that if execution starts in any state satisfying P and C terminates, then the resulting state satisfies Q . Hoare triples therefore provide a compact way to specify program behavior by relating assumptions before execution to guarantees after execution. In our gamegen-verification setting, we use this structure in a Hoare-style sense: the command component C is specialized to a bounded game-interaction sequence $\mathbf{a}$.

4 GameGen-Verifier

GameGen-Verifier introduces a state-grounded verification workflow (Figure 3, Section 4.1), proposes a parameter-based state construction and runtime injection mechanism to make keypoint verification executable (Section 4.2), and implements GGV-HARNESS, a scalable agentic verification harness for concurrency management, runtime isolation, and fault recovery (Section 4.3).

4.1 Verifiable Keypoint Extraction and Attribution

In LLM-generated games, the natural-language specification defines what the game should be. We observe that many specification-level failures can be exposed by a sparse set of critical behavioral conditions, often located at cross-system interactions such as collision and physics responses, inventory or resource updates, scoring and reward settlement, and progression gates. We therefore extract these conditions as verifiable keypoints and test them against the generated implementation, using their verdicts to derive specification-level judgments without traversing the full state space.

To make this notion precise, we define a *verifiable keypoint* as a natural-language Hoare-style triple $\varphi = (P, \mathbf{a}, Q)$, where P is a precondition over game states, $\mathbf{a}$ is a bounded interaction sequence, and Q is the expected postcondition. Each keypoint must satisfy three conditions: **(C1) Constructibility**: P describes a directly constructible game state; **(C2) Boundedness**: $\mathbf{a}$ involves a finite, short sequence of actions; **(C3) Verifiability**: Q is unambiguous enough to admit an operational PASS/FAIL verdict.

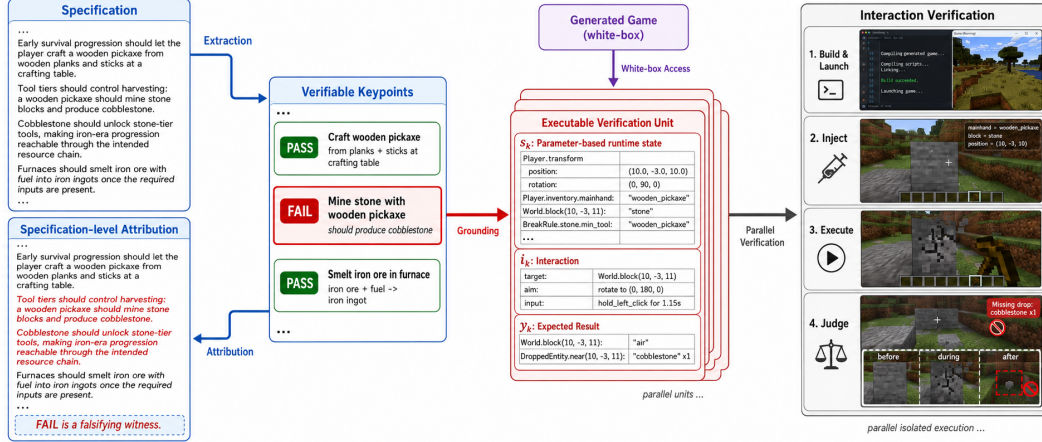


Figure 3: GameGen-Verifier extracts verifiable keypoints from a natural-language specification and grounds each into an executable verification unit comprising a parameter-based runtime state s_k , a bounded interaction i_k , and an expected outcome y_k , constructed via white-box access to the generated game. Each unit is executed through four steps: build and launch, inject the target state, execute the bounded interaction, and judge the observed outcome. FAIL verdicts propagate back to the specification as falsifying witnesses.

We use an LLM agent to extract candidate verifiable keypoints from the specification under the C1–C3 constraints, representing each accepted keypoint as a triple (P, a, Q) . This yields a finite set of verifiable keypoints from each game specification, framing correctness as a local, bounded property rather than a global trajectory property.

A verifiable keypoint as defined above is still a natural-language assertion: it states what should hold, but is not directly runnable. To make it executable against a concrete game implementation, we introduce a *verification unit* $u_k = (s_k, i_k, y_k)$ as the concrete runtime grounding of a keypoint. Compared with the keypoint triple (P, a, Q) , the verification unit grounds each natural-language component into an implementation-specific counterpart. The state s_k instantiates the precondition P via our parameter-based state construction mechanism (Section 4.2). Given s_k , the interaction a is grounded into an executable instruction i_k , and the postcondition Q is operationalized into a concrete expected outcome y_k . A single verifiable keypoint may be grounded into multiple verification units to cover distinct boundary conditions (e.g., testing collision under varying positions and velocities). Each unit is an independent, bounded evaluation: the verifier launches an isolated game instance, injects constructed state s_k into the runtime, executes i_k , and collects runtime evidence, including visual traces (e.g., screenshots) and internal runtime signals (e.g., logs or exposed state). The bounded evidence is judged against y_k by a vision-language model (VLM) judge, supplemented by programmatic assertions when the expected outcome can be expressed as predicates over exposed runtime state or logs. A unit receives a FAIL verdict if the generated game fails to build or launch, fails to realize the injected state after the harness’s retry procedure, fails to complete the bounded interaction, or produces behavior inconsistent with the expected outcome. Each verification unit is self-contained, eliminating inter-unit dependencies and naturally enabling parallel execution. This state-grounded formulation avoids the reachability problem of ordinary gameplay, reducing unreliable gameplay to a finite collection of parallelizable short-horizon verifications.

The specification is the scoring interface for both human and automated verification, so GameGen-Verifier closes the loop from specification to keypoints and back: each keypoint verdict is attributed to the specification element it covers and aggregated into a specification-level judgment. This specification-level aggregation follows a falsification-oriented scoring rule. A failed keypoint is treated as a falsifying witness for the specification element it covers, and the failure is propagated to downstream elements whose satisfaction depends on that behavior. After failure propagation, the remaining specification elements are assigned PASS. This means that no executed keypoint has exposed a violation of those elements, not that the generated game is formally proven correct on them.

GameGen-Verifier accordingly exports results at two levels. *Keypoint-level results* record each verifiable keypoint’s PASS/FAIL verdict evaluated independently via constructed states. *Specification-level results* aggregate keypoint verdicts back to specification elements.

4.2 Parameter-Based State Construction and Runtime Injection

To ground a verifiable keypoint into an executable verification unit $u_k = (s_k, i_k, y_k)$, the verifier must instantiate a target state s_k satisfying the keypoint precondition P . For games, this is challenging because states are large, continuous, path-dependent, and often reachable only after long interaction histories. If the verifier must reach s_k through gameplay, verification again becomes the long-horizon exploration problem that GameGen-Verifier is designed to avoid. We therefore reframe this prerequisite from a *reachability problem* (whether there exists an interaction trajectory from s_0 to a state satisfying P) to a *configuration problem*: directly construct a target state satisfying P from the game’s parameter structure.

Our key insight is that *LLM-generated games offer an asymmetry unavailable in traditional game evaluation*: the verifier has access not only to the deployed game, but also to the full implementation source produced by the generator. While the deployed game exposes only an interactive surface, the source code reveals the parameters governing its internal state. Since interactive software can be viewed as a data-driven state machine, a target game state can be characterized by setting the relevant entities, variables, flags, and relations. GameGen-Verifier exploits this structure by synthesizing state snapshots that satisfy each target precondition and injecting them into the runtime, localizing verification to a controlled starting point.

For example, instead of requiring a verifier to discover a boss fight through exploration, GameGen-Verifier can directly construct a state in which the boss is spawned, the player is placed in the correct arena, quest flags are advanced, and combat parameters are set to the desired phase. This converts dynamic exploration into static state construction. This distinction is especially important for games and other long-horizon interactive software, whereas many web tasks expose short, discrete, and easily reachable states that do not require parameter-based construction.

To put this construction into practice, the verifier must instantiate synthesized states directly in the runtime environment. Existing checkpointing or save/load mechanisms are often engine-specific, incomplete, or otherwise unsuitable for systematic evaluation. We therefore formalize an engine-agnostic injection contract: the verification harness requires access to a *runtime state patching* mechanism, i.e., programmatic modification of the game’s internal state after launch without replaying interaction history. This requires two basic capabilities: controlling *what entities exist* in the running scene (e.g., loading a level, spawning or removing objects) and controlling *what values they hold* (e.g., setting health, advancing quest flags, or repositioning characters). The constructed state need not reproduce transient execution context such as in-flight coroutines or animation frames exactly. It only needs to be *logically equivalent* with respect to the keypoint precondition and expected outcome, since verification postconditions are defined over observable behavior.

Fortunately, web-stack games often expose mutable JavaScript state through the browser runtime or injected instrumentation, making runtime state patching comparatively straightforward [9]. Our experiments therefore focus on web-stack games. As discussed in Section 3, mainstream engines such as Godot, Unity, and Unreal Engine are outside our experimental scope because LLM-based generation for these engines remains an open problem. We discuss them as contract-compatible extensions, since they also support runtime state patching mechanisms [14, 16, 38]. We discuss engine-specific mappings in Appendix B.

4.3 Scalable Agentic Verification Harness

A natural implementation of GameGen-Verifier is to run the entire workflow inside a modern coding-agent session, such as Claude Code or Codex. In this design, a parent agent is equipped with multiple task-specific skills, dispatches verification units to built-in subagents, and later merges the returned results. However, this agent-native harness is fragile as the execution substrate for the full workflow. GameGen-Verifier must evaluate many units across games, keypoints, and repeated trials, while current sub-agent mechanisms provide limited support for parallel scheduling, explicit concurrency control, fault-tolerant recovery, and runtime isolation. The core systems challenge is therefore reliable execution management for a highly parallel verification workload.

We implement GGV-HARNESS as a two-layer agentic verification harness that separates orchestration from local reasoning. The Python *orchestration layer* maintains the unit queue, enforces concurrency and rate limits, applies timeout and retry policies, persists checkpoints, and aggregates verdicts. The *worker layer* consists of short-lived LLM agent calls, each scoped to one self-contained verification

unit $u_k = (s_k, i_k, y_k)$, such as constructing an injectable state, generating an interaction script, or judging observed behavior from runtime evidence. Because units are independent, workers can be invoked statelessly: each processes one verification unit and returns structured evidence and a verdict. The orchestrator can then dispatch units concurrently, retry failed or timed-out units, and resume interrupted runs from checkpoints. To provide runtime isolation, each worker launches an independent game instance, preventing interference through shared state such as random seeds, timers, or browser-level singletons. Together, this design retains coding agents for local reasoning while providing the concurrency control, runtime isolation, and fault recovery needed for large-scale, repeatable verification.

5 Evaluation

We validate GameGen-Verifier through two sets of experiments: a verification study against human judgment and a harness study that isolates the scalability and stability benefits of GGV-HARNESS.

5.1 Dataset

We build VERIGAME, a game specification dataset for automated verification of LLM-generated games, since no such dataset is currently available. VERIGAME contains 100 game-generation tasks spanning seven genres (Action, Adventure, Casual, Puzzle, Simulation, Strategy, and Board), each consisting of an anonymized game identifier and a natural-language specification. The dataset is constructed through a strict sourcing, specification synthesis, and human refinement pipeline. We provide full details of the data synthesis pipeline in Appendix E.

5.2 Experimental Setup

Game-generation Stack. As discussed in Section 3, current LLM-based game generation for mainstream engines such as Godot, Unity, and Unreal Engine remains immature. These engines are therefore outside the experimental scope of this work. We therefore evaluate web-stack games implemented in JavaScript, TypeScript, or HTML, using each game’s specification from VERIGAME as input. Games are generated by the coding agent: Claude Code v2.1.72 (Opus 4.6, high effort).

Baselines. We compare against two Agent-as-a-Verifier (AaaV) variants [25, 29–31, 43], the current state-of-the-art paradigm for evaluating LLM-generated interactive software, which has been widely adopted for web application verification. *AaaV-Direct* is the standard prompt-only baseline: an evaluation agent is prompted to directly play the generated game and verify the observed behavior against the full natural-language specification. This setting reflects the common use of AaaV, but it may omit hard-to-reach or low-salience specification elements. We therefore also evaluate *AaaV-CE* (coverage-enforced AaaV), which wraps the same direct verifier in a todo-list-style coverage harness over the specification elements. After each agent response, a judge checks whether every specification element has an explicit verdict. If any element is missing or ambiguous, the harness forces the agent to continue until the list is complete or the retry budget is exhausted. This makes AaaV-CE a stronger AaaV baseline than AaaV-Direct. The full AaaV-Direct prompt is provided in Appendix C. All methods produce specification-level verdicts, enabling direct comparison.

5.3 Alignment with Human Judgment

This experiment evaluates whether GameGen-Verifier produces verdicts that agree with human judgment and whether it offers efficiency gains over AaaV-style verification once explicit specification-level coverage is required. We evaluate GameGen-Verifier, AaaV-Direct, and AaaV-CE under three underlying-agent settings: Codex CLI v0.125.0 (GPT-5.4 xhigh), Claude Code v2.1.72 (Opus 4.6, high effort), and Claude Code v2.1.72 (Kimi 2.6 Coding). In GameGen-Verifier, the agent serves as the GGV-HARNESS worker. AaaV-Direct uses the same agent to directly play the generated game and executes verification against the full specification. AaaV-CE places the same direct-verification agent inside the todo-list coverage harness, which repeatedly judges whether all specification elements have verdicts and forces continuation when the list is incomplete. For each specification in VERIGAME, we generate a web-stack implementation and run each automated method 5 times for each generated game under each setting. As discussed in Section 4.1, the specification serves as a unified scoring interface: all automated methods are ultimately scored on the same specification elements. The

Table 1: Results of alignment with human judgment across different verification methods and agent settings. $Prec@5$ penalizes false PASS predictions, $Rec@5$ penalizes missed human-PASS elements, and $Time@5$ reports average wall-clock time over five runs.

Backend	Method	Acc@5	Prec@5	Rec@5	F1@5	Time@5
Codex	AaaV-Direct	0.011	0.667	0.009	0.018	387s
Codex	AaaV-CE	0.588	0.968	0.484	0.645	7356s
Codex	GameGen-Verifier	0.922	0.912	1.000	0.954	443s
Claude Code (Opus)	AaaV-Direct	0.013	0.714	0.011	0.022	552s
Claude Code (Opus)	AaaV-CE	0.576	0.935	0.475	0.630	8245s
Claude Code (Opus)	GameGen-Verifier	0.902	0.892	1.000	0.943	513s
Claude Code (Kimi)	AaaV-Direct	0.033	0.812	0.021	0.040	537s
Claude Code (Kimi)	AaaV-CE	0.510	0.923	0.522	0.667	9072s
Claude Code (Kimi)	GameGen-Verifier	0.866	0.893	0.962	0.926	614s

human reference and GameGen-Verifier produce binary specification-level labels, PASS or FAIL. The AaaV baselines may additionally return UNVERIFIED when a specification element is not verified or the agent provides no supporting evidence for a binary verdict.

Human Reference. As the human reference, 3 expert human evaluators independently play each generated game and assess its correctness against the specification. Each evaluator plays the game and marks which specification elements the implementation fails to satisfy based on observed gameplay behavior. The final human verdict is determined by majority vote across the evaluators. This produces the same specification-level verdicts as these automated methods.

Metrics. All automatic metrics are computed over 5 repeated runs, using PASS as the positive class. We report five metrics against the human reference: $Acc@5$ measures overall label agreement, $Prec@5$ measures the correctness of predicted PASS labels, $Rec@5$ measures coverage of human-PASS elements, $F1@5$ is the harmonic mean of precision and recall, and $Time@5$ measures wall-clock execution time. Appendix A gives the full formulas, including how AaaV UNVERIFIED outcomes are scored.

Results. Table 1 shows that GameGen-Verifier is more accurate and faster than AaaV-style verification. GameGen-Verifier achieves $Acc@5/F1@5$ of 0.922/0.954, 0.902/0.943, and 0.866/0.926 under the Codex, Claude Code (Opus), and Claude Code (Kimi) settings, respectively. AaaV-Direct has very low accuracy because it often terminates after checking only a small, easy subset of the specification, while AaaV-CE reaches at most 0.588 $Acc@5$, far below GameGen-Verifier. Our trajectory inspection shows the reason: it can often verify directly observable, short-horizon specification elements, but it still struggles with elements that require sustained gameplay, precise state reachability, or multi-step causal attribution.

Across Codex, Claude Code (Opus), and Claude Code (Kimi), GameGen-Verifier consistently outperforms AaaV-CE; Codex and Claude Code (Opus) are closely matched, and Claude Code (Kimi) remains substantially above its AaaV-CE counterpart. The precision/recall pattern, especially GameGen-Verifier’s high recall (1.000 under Codex and Claude Code (Opus), 0.962 under Claude Code (Kimi)), also reflects the falsification-oriented design of GameGen-Verifier: keypoint extraction can lose coverage, but observed keypoint violations provide strong evidence of real specification failures. Finally, GameGen-Verifier substantially reduces wall-clock time by up to $16.6\times$ compared with AaaV-CE by converting verification into local, bounded units that can be executed in parallel.

5.4 Agentic Verification Harness

This experiment evaluates the scalability and execution guarantees of GGV-HARNESS as an execution harness for state-grounded verification.

We compare GGV-HARNESS with *Agent-Native Harness*, a strong agent-native execution baseline described in Section 4.3. Agent-Native Harness implements the same state-grounded verification workflow entirely inside a modern coding-agent session. The parent agent is equipped with task-specific verification skills for keypoint grounding, state construction and injection, interaction-script generation, runtime evidence collection, and verdict aggregation. To exploit unit independence, the

Table 2: Harness-level scalability and execution-guarantee comparison. GGV-HARNESS is the only execution substrate that combines low wall-clock time with explicit concurrency control, runtime isolation, and fault recovery.

Method	Backend	Time	Concurrency Control	Runtime Isolation	Fault Recovery
Agent-Native Harness	Codex	1293s	Partial	×	×
GGV-HARNESS	Codex	457s	✓	✓	✓
Agent-Native Harness	Claude Code	1594s	Partial	×	×
GGV-HARNESS	Claude Code	536s	✓	✓	✓

parent agent uses its built-in subagent mechanism to dispatch multiple verification units in parallel, waits for the returned evidence and verdicts, and merges them into a game-level report. To isolate the effect of the harness design, we run both harnesses under two settings: Codex CLI v0.125.0 (GPT-5.4 xhigh) and Claude Code v2.1.72 (Opus 4.6, high effort), and keep the verification workload and evaluation budget identical across methods within each setting: the same generated games, extracted keypoints, retry budget, and timeout policy are used for both methods. We then compare how each harness controls concurrency, isolates runtime executions, and recovers from failures.

Metrics. We report wall-clock interaction time for completed verification runs and separately record whether each execution mode provides the core systems guarantees required by Section 4.3: concurrency management, runtime isolation, and fault recovery. Both methods are evaluated under the same timeout budget.

Results. Table 2 shows that scalable verification requires more than invoking more agent workers. Agent-Native Harness can invoke subagents, but execution is still coordinated through the parent agent session rather than through an explicit scheduler with controlled resource limits. As a result, runtime isolation and recovery from partial failures are best-effort behaviors rather than execution guarantees. By contrast, GGV-HARNESS reduces average wall-clock time by up to 66.4% relative to Agent-Native Harness while adding the guarantees needed for repeatable evaluation. Each worker agent is short-lived and bound to one verification unit, so failed or timed-out units can be retried without invalidating the rest of the game-level run. Per-unit runtime isolation also prevents interference through shared browser state, timers, random seeds, or mutated game globals.

6 Limitations

GameGen-Verifier currently operates on web-stack games (JavaScript, TypeScript, HTML) and has not been validated on Godot, Unity, or Unreal Engine, where LLM-based generation pipelines remain immature, making engine-native adapters a natural next step once such pipelines mature. Keypoint extraction is also not guaranteed to be complete, and state construction requires white-box access to the implementation to identify its parameter structure, which limits applicability to games whose internal state resists programmatic inspection. Finally, GameGen-Verifier is falsification-oriented: a passing verdict means no executed keypoint exposed a violation, not that the game is formally proven correct.

7 Conclusion

We presented GameGen-Verifier, a verification paradigm for LLM-generated games via runtime state injection that decomposes a natural-language specification into independent, parallelizable verification units, each grounded in a concrete runtime state patch and checked through bounded interaction. GGV-HARNESS makes this practical at scale through explicit concurrency control, runtime isolation, and fault recovery. On VERIGAME, our dataset of 100 LLM-generated games across seven genres, GameGen-Verifier substantially improves agreement with expert human judgments while reducing wall-clock time relative to coverage-enforced Agent-as-a-Verifier baselines, and we view this approach as a practical step toward reliable game generation infrastructure.

References

- [1] Alex Ermolov. Godogen. <https://github.com/htdt/godogen>, 2026.
- [2] Anthropic. Claude code. <https://github.com/anthropics/claude-code>. Accessed: 2026.
- [3] Anuttacon. Anuttacon: AI-driven interactive entertainment. <https://anuttacon.com>, 2024.
- [4] Apple. Games for iphone on the app store. <https://apps.apple.com/us/iphone/games>, 2026. Accessed: 2026.
- [5] Sinan Ariyurek, Aysu Betin-Can, and Elif Surer. Automated video game testing using synthetic and human-like agents. *IEEE Transactions on Games*, 13(1):50–67, 2021.
- [6] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- [7] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- [8] ChiR24. Unreal engine mcp server. https://github.com/ChiR24/Unreal_mcp, 2026. Accessed: 2026.
- [9] Chrome DevTools Protocol. Chrome devtools protocol: Runtime domain. <https://chromedevtools.github.io/devtools-protocol/v8/Runtime/>, 2026. Accessed: 2026.
- [10] CoplayDev. Mcp for unity. <https://github.com/CoplayDev/unity-mcp>, 2026. Accessed: 2026.
- [11] ee0pdt. Godot mcp (model context protocol). <https://github.com/ee0pdt/Godot-MCP>, 2026. Accessed: 2026.
- [12] Epic Games. Actors in unreal engine. <https://dev.epicgames.com/documentation/en-us/unreal-engine/actors-in-unreal-engine>, 2026. Accessed: 2026.
- [13] Epic Games. Unreal object handling. <https://dev.epicgames.com/documentation/en-us/unreal-engine/unreal-object-handling>, 2026. Accessed: 2026.
- [14] Epic Games. Uworld::spawnactor. <https://dev.epicgames.com/documentation/en-us/unreal-engine/API/Runtime/Engine/Engine/UWorld/SpawnActor/1>, 2026. Accessed: 2026.
- [15] Roberto Gallotta, Graham Todd, Marvin Zammit, Sam Earle, Antonios Liapis, Julian Togelius, and Georgios N. Yannakakis. Large language models and games: A survey and roadmap. *IEEE Transactions on Games*, pages 1–18, 2024.
- [16] Godot Engine Documentation. Nodes and scene instances. https://docs.godotengine.org/en/4.5/tutorials/scripting/nodes_and_scene_instances.html, 2026. Accessed: 2026.
- [17] Godot Engine Documentation. Packedscene: Godot engine documentation. https://docs.godotengine.org/en/4.5/classes/class_packedscene.html, 2026. Accessed: 2026.
- [18] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, et al. Measuring coding challenge competence with apps. *arXiv preprint arXiv:2105.09938*, 2021.
- [19] C. A. R. Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12(10):576–580, 1969.
- [20] Jiale Hong, Xuefeng Liu, and Weinan E. Game development as human-llm interaction, 2024.

- [21] Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. Metagpt: Meta programming for a multi-agent collaborative framework, 2024.
- [22] Yilei Jiang, Jinyuan Hu, Qianyin Xiao, Yaozhi Zheng, Ruize Ma, Kaituo Feng, Jiaming Han, Tianshuo Peng, Kaixuan Fan, Manyuan Zhang, and Xiangyu Yue. OpenGame: Open agentic coding for games, 2026.
- [23] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues?, 2024.
- [24] Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven C. H. Hoi. CodeRL: Mastering code generation through pretrained models and deep reinforcement learning. In *Advances in Neural Information Processing Systems*, 2022.
- [25] Xinping Lei, Xinyu Che, Junqi Xiong, Chenchen Zhang, Yukai Huang, Chenyu Zhou, Haoyang Huang, Minghao Liu, Letian Zhu, Hongyi Ye, Jinhua Hao, Ken Deng, Zizheng Zhan, Han Li, Dailin Li, Yifan Yao, Ming Sun, Zhaoxiang Zhang, and Jiaheng Liu. Webcompass: Towards multimodal web coding evaluation for code language models, 2026.
- [26] Bowen Li, Wenhan Wu, Ziwei Tang, Lin Shi, John Yang, Jinyang Li, Shunyu Yao, Chen Qian, Binyuan Hui, Qicheng Zhang, Zhiyin Yu, He Du, Ping Yang, Dahua Lin, Chao Peng, and Kai Chen. Prompting large language models to tackle the full software development lifecycle: A case study, 2024.
- [27] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dalgleish, et al. Competition-level code generation with AlphaCode. *Science*, 378(6624):1092–1097, 2022.
- [28] Jiate Liu, Yiqin Zhu, Kaiwen Xiao, Qiang Fu, Xiao Han, Wei Yang, and Deheng Ye. RLTF: Reinforcement learning from unit test feedback. *Transactions on Machine Learning Research*, 2023.
- [29] Yuxuan Lu, Bingsheng Yao, Hansu Gu, Jing Huang, Zheshen Jessie Wang, Yang Li, Jiri Gesi, Qi He, Toby Jia-Jun Li, and Dakuo Wang. Uxagent: An llm agent-based usability testing framework for web design. In *Proceedings of the Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*, CHI EA '25, pages 1–12. ACM, April 2025.
- [30] Zimu Lu, Yunqiao Yang, Houxing Ren, Haotian Hou, Han Xiao, Ke Wang, Weikang Shi, Aojun Zhou, Mingjie Zhan, and Hongsheng Li. Webgen-bench: Evaluating llms on generating interactive and functional websites from scratch. *arXiv preprint arXiv:2505.03733*, 2025.
- [31] MiniMax. Minimax m2.1: Significantly enhanced multi-language programming, built for real-world complex tasks. Release note / model documentation, 2025. Model repository: <https://github.com/MiniMax-AI/MiniMax-M2.1>.
- [32] OpenAI. Codex cli. <https://github.com/openai/codex>. Accessed: 2026.
- [33] OpenCode Contributors. Opencode. <https://github.com/anomalyco/opencode>. Accessed: 2026.
- [34] Cristiano Politowski, Fabio Petrillo, and Yann-Gael Gueheneuc. A survey of video game testing. In *2021 IEEE/ACM International Conference on Automation of Software Test (AST)*, pages 90–99, 2021.
- [35] Samantha Stahlke, Atiya Nova, and Pejman Mirza-Babaei. Artificial playfulness: A tool for automated agent-based playtesting. In *Extended Abstracts of the 2019 CHI Conference on Human Factors in Computing Systems*, CHI EA '19, pages 1–6, New York, NY, USA, 2019. Association for Computing Machinery.

- [36] Kimi Team, Tongtong Bai, Yifan Bai, Yiping Bao, S. H. Cai, Yuan Cao, Y. Charles, H. S. Che, Cheng Chen, Guanduo Chen, Huarong Chen, Jia Chen, Jiahao Chen, Jianlong Chen, Jun Chen, Kefan Chen, Liang Chen, Ruijue Chen, Xinhao Chen, Yanru Chen, Yanxu Chen, Yicun Chen, Yimin Chen, Yingjiang Chen, Yuankun Chen, Yujie Chen, Yutian Chen, Zhirong Chen, Ziwei Chen, Dazhi Cheng, Minghan Chu, Jiale Cui, Jiaqi Deng, Muxi Diao, Hao Ding, Mengfan Dong, Mengnan Dong, Yuxin Dong, Yuhao Dong, Angang Du, Chenzhuang Du, Dikang Du, Lingxiao Du, Yulun Du, Yu Fan, Shengjun Fang, Qiulin Feng, Yichen Feng, Garimugai Fu, Kelin Fu, Hongcheng Gao, Tong Gao, Yuyao Ge, Shangyi Geng, Chengyang Gong, Xiaochen Gong, Zhuoma Gongque, Qizheng Gu, Xinran Gu, Yicheng Gu, Longyu Guan, Yuanying Guo, Xiaoru Hao, Weiran He, Wenyang He, Yunjia He, Chao Hong, Hao Hu, Jiaxi Hu, Yangyang Hu, Zhenxing Hu, Ke Huang, Ruiyuan Huang, Weixiao Huang, Zhiqi Huang, Tao Jiang, Zhejun Jiang, Xinyi Jin, Yu Jing, Guokun Lai, Aidi Li, C. Li, Cheng Li, Fang Li, Guanghe Li, Guanyu Li, Haitao Li, Haoyang Li, Jia Li, Jingwei Li, Junxiong Li, Lincan Li, Mo Li, Weihong Li, Wentao Li, Xinhang Li, Xinhao Li, Yang Li, Yanhao Li, Yiwei Li, Yuxiao Li, Zhaowei Li, Zheming Li, Weilong Liao, Jiawei Lin, Xiaohan Lin, Zhishan Lin, Zichao Lin, Cheng Liu, Chenyu Liu, Hongzhang Liu, Liang Liu, Shaowei Liu, Shudong Liu, Shuran Liu, Tianwei Liu, Tianyu Liu, Weizhou Liu, Xiangyan Liu, Yangyang Liu, Yanming Liu, Yibo Liu, Yuanxin Liu, Yue Liu, Zhengying Liu, Zhongnuo Liu, Enzhe Lu, Haoyu Lu, Zhiyuan Lu, Junyu Luo, Tongxu Luo, Yashuo Luo, Long Ma, Yingwei Ma, Shaoguang Mao, Yuan Mei, Xin Men, Fanqing Meng, Zhiyong Meng, Yibo Miao, Mingqing Ni, Kun Ouyang, Siyuan Pan, Bo Pang, Yuchao Qian, Ruoyu Qin, Zeyu Qin, Jiezhong Qiu, Bowen Qu, Zeyu Shang, Youbo Shao, Tianxiao Shen, Zhennan Shen, Juanfeng Shi, Lidong Shi, Shengyuan Shi, Feifan Song, Pengwei Song, Tianhui Song, Xiaoxi Song, Hongjin Su, Jianlin Su, Zhaochen Su, Lin Sui, Jinsong Sun, Junyao Sun, Tongyu Sun, Flood Sung, Yunpeng Tai, Chuning Tang, Heyi Tang, Xiaojuan Tang, Zhengyang Tang, Jiawen Tao, Shiyuan Teng, Chaoran Tian, Pengfei Tian, Ao Wang, Bowen Wang, Chensi Wang, Chuang Wang, Congcong Wang, Dingkun Wang, Dinglu Wang, Dongliang Wang, Feng Wang, Hailong Wang, Haiming Wang, Hengzhi Wang, Huaqing Wang, Hui Wang, Jiahao Wang, Jinhong Wang, Jiuzheng Wang, Kaixin Wang, Linian Wang, Qibin Wang, Shengjie Wang, Shuyi Wang, Si Wang, Wei Wang, Xiaochen Wang, Xinyuan Wang, Yao Wang, Yejie Wang, Yipu Wang, Yiqin Wang, Yucheng Wang, Yuzhi Wang, Zhaoji Wang, Zhaowei Wang, Zhengtao Wang, Zhexu Wang, Zihan Wang, Zizhe Wang, Chu Wei, Ming Wei, Chuan Wen, Zichen Wen, Chengjie Wu, Haoning Wu, Junyan Wu, Rucong Wu, Wenhao Wu, Yuefeng Wu, Yuhao Wu, Yuxin Wu, Zijian Wu, Chenjun Xiao, Jin Xie, Xiaotong Xie, Yuchong Xie, Yifei Xin, Bowei Xing, Boyu Xu, Jianfan Xu, Jing Xu, Jinjing Xu, L. H. Xu, Lin Xu, Suting Xu, Weixin Xu, Xinbo Xu, Xinran Xu, Yangchuan Xu, Yichang Xu, Yuemeng Xu, Zelai Xu, Ziyao Xu, Junjie Yan, Yuzi Yan, Guangyao Yang, Hao Yang, Junwei Yang, Kai Yang, Ningyuan Yang, Ruihan Yang, Xiaofei Yang, Xinlong Yang, Ying Yang, Yi Yang, Yi Yang, Zhen Yang, Zhilin Yang, Zonghan Yang, Haotian Yao, Dan Ye, Wenjie Ye, Zhuorui Ye, Bohong Yin, Chengzhen Yu, Longhui Yu, Tao Yu, Tianxiang Yu, Enming Yuan, Mengjie Yuan, Xiaokun Yuan, Yang Yue, Weihao Zeng, Dunyuan Zha, Haobing Zhan, Dehao Zhang, Hao Zhang, Jin Zhang, Puqi Zhang, Qiao Zhang, Rui Zhang, Xiaobin Zhang, Y. Zhang, Yadong Zhang, Yangkun Zhang, Yichi Zhang, Yizhi Zhang, Yongting Zhang, Yu Zhang, Yushun Zhang, Yutao Zhang, Yutong Zhang, Zheng Zhang, Chenguang Zhao, Feifan Zhao, Jinxian Zhao, Shuai Zhao, Xiangyu Zhao, Yikai Zhao, Zijia Zhao, Huabin Zheng, Ruihan Zheng, Shaojie Zheng, Tengyang Zheng, Junfeng Zhong, Longguang Zhong, Weiming Zhong, M. Zhou, Runjie Zhou, Xinyu Zhou, Zaida Zhou, Jinguo Zhu, Liya Zhu, Xinhao Zhu, Yuxuan Zhu, Zhen Zhu, Jingze Zhuang, Weiyu Zhuang, Ying Zou, and Xinxing Zu. Kimi k2.5: Visual agentic intelligence, 2026.
- [37] Unity Technologies. Unity scripting api: `GameObject.GetComponent`. <https://docs.unity3d.com/ScriptReference/GameObject.GetComponent.html>, 2026. Accessed: 2026.
- [38] Unity Technologies. Unity scripting api: `Object.Instantiate`. <https://docs.unity3d.com/ScriptReference/Object.Instantiate.html>, 2026. Accessed: 2026.
- [39] Valve. Steam store. <https://store.steampowered.com/>, 2026. Accessed: 2026.
- [40] Valve. Steam tags. <https://partner.steamgames.com/doc/store/tags>, 2026. Accessed: 2026.

- [41] Stefan Varvaressos, Kim Lavoie, Sarah Gaboury, and Sylvain Hallé. Automated bug finding in video games: A case study for runtime monitoring. *Computers in Entertainment*, 15(1):1–28, 2017.
- [42] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W White, Doug Burger, and Chi Wang. Autogen: Enabling next-gen llm applications via multi-agent conversation, 2023.
- [43] Xiaomi LLM-Core Team. Mimo-v2-flash technical report. *arXiv preprint arXiv:2601.02780*, 2026.
- [44] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, 2023.
- [45] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena. *arXiv preprint arXiv:2306.05685*, 2023.

Broader Impact and Ethical Considerations

Broader impact. This work develops a verification paradigm for LLM-generated games, contributing to the infrastructure needed to make game generation reliable and measurable. On the positive side, scalable verification lowers the barrier to game creation by enabling non-expert users to trust that generated games behave as intended, potentially democratizing access to game development. On the negative side, more reliable verification could accelerate the automated generation of low-quality or derivative game content at scale. We do not foresee significant risks beyond those already associated with LLM-based code generation in general.

Ethical considerations. VERIGAME is a dataset designed for benchmarking automated verification of LLM-generated games, not a redistribution of existing commercial games. Its examples are released as anonymized natural-language specifications that describe gameplay mechanics and verification-relevant rules at an abstract functional level. We do not redistribute proprietary game code, assets, or other expressive media from the source games; game identities are anonymized and references to original titles are removed before generation and evaluation. These measures are intended to reduce copyright and trademark risks while preserving the research value of evaluating whether LLM-generated games satisfy functional specifications; our release should not be used to clone or market substitutes for existing games. The dataset and verification framework are intended for research on reliable game generation, automated evaluation, and specification-grounded feedback, not to reproduce protected commercial content or evade the rights of game developers. If concerns about a released specification are raised by rights holders, we will review the example and remove or revise it when appropriate.

Code and Dataset Licenses

Codebase. The verification framework will be released under the MIT License upon acceptance.

Datasets. The VERIGAME dataset will be released under CC BY 4.0 upon acceptance.

Appendix

A Metric Definitions

All alignment metrics are computed at the specification-element level against a binary human reference label $r \in \{\text{PASS}, \text{FAIL}\}$. GameGen-Verifier also returns binary specification-level labels. The AaaV baselines may additionally return UNVERIFIED when a specification element is not checked or the evidence is insufficient. We treat PASS as the positive class.

We define the extended confusion counts as

	$r = \text{PASS}$	$r = \text{FAIL}$
$\hat{y} = \text{PASS}$	TP	FP
$\hat{y} = \text{FAIL}$	FN	TN
$\hat{y} = \text{UNVERIFIED}$	Unverified ₊	Unverified ₋

and let $n = TP + FP + FN + TN + \text{Unverified}_+ + \text{Unverified}_-$. For binary-output methods such as GameGen-Verifier, $\text{Unverified}_+ = \text{Unverified}_- = 0$, and the metrics reduce to the standard binary definitions:

$$\text{Acc} = \frac{TP + TN}{TP + FP + FN + TN}, \quad \text{Prec} = \frac{TP}{TP + FP}, \quad \text{Rec} = \frac{TP}{TP + FN},$$

$$\text{F1} = \frac{2 \cdot \text{Prec} \cdot \text{Rec}}{\text{Prec} + \text{Rec}}.$$

For AaaV-Direct and AaaV-CE, UNVERIFIED outcomes are retained rather than removed from scoring. They count as non-agreements for accuracy, and UNVERIFIED outcomes on human-PASS elements count as missed positives for recall:

$$\text{Acc} = \frac{TP + TN}{n}, \quad \text{Prec} = \frac{TP}{TP + FP}, \quad \text{Rec} = \frac{TP}{TP + FN + \text{Unverified}_+},$$

$$\text{F1} = \frac{2 \cdot \text{Prec} \cdot \text{Rec}}{\text{Prec} + \text{Rec}}.$$

Time@5 is the wall-clock execution time averaged over the five repeated runs.

B Engine-Specific Runtime State Patching

As discussed in Section 4.2, our verification paradigm requires the generated implementation to support runtime state patching: the ability to programmatically control what entities exist in a running scene and what values they hold [9, 14, 16, 38]. In this appendix, we describe how this contract maps in principle to representative APIs across four runtimes: JavaScript (web stack), Godot, Unity, and Unreal Engine. The examples below are illustrative mappings rather than engine-native implementations used in our experiments.

To make the discussion concrete, we use a running example throughout: constructing a boss fight state in which the boss is spawned in a designated arena, the player is positioned nearby with specific health and inventory, quest flags are advanced to the boss phase, and combat parameters are initialized.

B.1 JavaScript / Web Stack

Web-stack games (e.g., those built with Phaser, Three.js, or vanilla Canvas/DOM) run in a browser environment where game state is typically held in mutable JavaScript objects and can often be inspected or modified through the JavaScript runtime [9]. Runtime state patching is comparatively straightforward when relevant state is exposed through the global scope, module exports, debug hooks, or lightweight instrumentation.

Listing 1: Runtime state patching in a web-stack game.

```
// Spawn boss in the arena
const boss = game.addEntity("BossEnemy", {
```

```

    position: { x: 500, y: 300 },
    archetype: "dragon",
    parent: game.scenes["bossArena"]
  });

  // Configure player and game state
  game.player.position = { x: 480, y: 350 };
  game.player.health = 100;
  game.player.inventory.push("legendary_sword");
  game.questManager.setFlag("boss_phase", true);
  game.combat.bossPhase = 2;
  game.combat.timer = 0;

```

Because JavaScript is dynamically typed and exposed runtime objects are mutable, LLM-generated web games often have less indirection between the evaluator and accessible game state. This makes web-stack games the most straightforward target for our verification paradigm, which is why our current experiments focus on this runtime.

B.2 Godot Engine

Godot organizes game state as a tree of Node objects (the scene tree), where each node has typed properties accessible via GDScript or the Godot API [16]. Runtime state patching maps onto Godot’s scene tree manipulation and property access.

Listing 2: Runtime state patching in Godot (GDScript).

```

# Load arena and spawn boss
var arena = load("res://scenes/BossArena.tscn").instantiate()
get_tree().current_scene.add_child(arena)

var boss = load("res://prefabs/Dragon.tscn").instantiate()
boss.position = Vector2(500, 300)
arena.add_child(boss)

# Configure player and game state
var player = get_node("/root/Main/Player")
player.position = Vector2(480, 350)
player.get_node("Health").current_hp = 100
player.get_node("Inventory").add_item("legendary_sword")

GameManager.quest_flags["boss_phase"] = true
GameManager.combat_phase = 2
GameManager.combat_timer = 0.0

```

Godot’s scene tree is accessible and mutable at runtime, and nodes can be instantiated, reparented, or removed programmatically [16, 17]. Notably, community MCP (Model Context Protocol) servers have been developed for Godot that expose engine operations such as scene tree inspection, node creation, and script editing to external AI agents [11]. However, these MCP integrations are still in an early stage of development.

B.3 Unity Engine

Unity organizes game state through GameObjects and Components [37]. Each GameObject lives in a scene hierarchy and carries components that define its behavior and data. Runtime state patching maps onto Unity’s programmatic scene management, object instantiation, and component property access [37, 38].

Listing 3: Runtime state patching in Unity (C#).

```

// Load arena scene, spawn boss
SceneManager.LoadScene("BossArena");

var bossPrefab = Resources.Load<GameObject>("Prefabs/Dragon");

```

```

var boss = Instantiate(bossPrefab,
    new Vector3(500, 0, 300), Quaternion.identity);
boss.transform.SetParent(
    GameObject.Find("ArenaRoot").transform);

// Configure player and game state
var player = GameObject.FindWithTag("Player");
player.transform.position = new Vector3(480, 0, 350);
player.GetComponent<Health>().currentHP = 100;
player.GetComponent<Inventory>().AddItem("legendary_sword");

GameManager.Instance.questFlags["boss_phase"] = true;
GameManager.Instance.combatPhase = 2;
GameManager.Instance.combatTimer = 0f;

```

Unity's Prefab system provides a natural mapping for controlling what entities exist: prefabs serve as archetypes that can be instantiated at arbitrary positions in the scene hierarchy. Controlling what values entities hold is supported through direct component field access or, for more generic injection, through C# reflection. Overall, Unity's runtime provides primitives for runtime state patching through scene management, object instantiation, and component access.

B.4 Unreal Engine 5

Unreal Engine organizes game state through Actors placed in a `UWorld`, with behavior defined by `UActorComponents` [12]. State patching maps onto Unreal's actor spawning, property system, and C++ or Blueprint scripting [14].

Listing 4: Runtime state patching in Unreal Engine 5 (C++).

```

// Spawn boss in the arena
FActorSpawnParameters SpawnParams;
SpawnParams.Owner = ArenaRoot;
ABossEnemy* Boss = GetWorld()->SpawnActor<ABossEnemy>(
    BossClass,
    FVector(500.f, 300.f, 0.f),
    FRotator::ZeroRotator,
    SpawnParams);

// Configure player and game state
APlayerCharacter* Player = Cast<APlayerCharacter>(
    UGameplayStatics::GetPlayerCharacter(this, 0));
Player->SetActorLocation(FVector(480.f, 350.f, 0.f));
Player->HealthComponent->CurrentHP = 100.f;
Player->InventoryComponent->AddItem(
    TEXT("legendary_sword"));

AMyGameState* GS = GetWorld()->GetGameState<AMyGameState>();
GS->QuestFlags.Add(TEXT("boss_phase"), true);
GS->CombatPhase = 2;
GS->CombatTimer = 0.f;

```

Unreal's property system (`UPROPERTY` macro) exposes component fields to both the editor and runtime reflection, enabling generic state patching without hardcoded field access [13]. Like Unity, Unreal's runtime supports programmatic control over the actor hierarchy and component state, providing primitives for runtime state patching.

B.5 Summary

These runtimes provide primitives that can support the two fundamental capabilities required by our injection contract: controlling what entities exist in a running scene and controlling what values they hold [9, 14, 16, 38]. The key difference lies in accessibility: web-stack games expose state with less indirection, while game engines require going through their respective object models (scene tree,

GameObject/Component, Actor/Component). Since Godot, Unity, and Unreal Engine do not yet support mature LLM-based game generation (community MCP servers for these engines have begun to appear but remain early-stage [8, 10, 11]), they are outside the scope of our current experiments. However, as demonstrated above, all three expose runtime state patching primitives, meaning our verification paradigm could in principle be extended to these engines with engine-specific adapters once their generation pipelines mature [14, 16, 38].

C Agent-as-a-Verifier Direct Baseline Prompt

We provide the user prompt used for the Agent-as-a-Verifier (AaaV) baseline below. The agent is given an anonymized game identifier, the corresponding specification path, and a target run identifier. The system prompt is empty, so the baseline executes purely from the user prompt without any additional skill scaffolding. The prompt template is:

Evaluate whether the game implementation is correct.

Hard requirements:

- Your current working directory is the generated implementation directory for anonymized game identifier '{game_id}'. The primary specification file is '{specification_path}'. If it is missing, say so explicitly in the result.
- You must choose your own testing path, but you must write a structured record of what you actually tested.
- Write 'baseline_eval_result.json' in the current working directory.
- All output text must be in English, including 'checks', 'findings', 'limitations', and 'summary'.
- Keep the evaluation bounded and pragmatic. Do not keep exploring once you already have decisive evidence for the final verdict.
- Required workflow:
 1. Read the full specification end-to-end. Enumerate every concrete specification element, where an element may be a sentence-level or clause-level requirement covering inputs, physics/timing, win conditions, lose conditions, state transitions, game rules, HUD/UI requirements, boundary conditions, or other categories.
 2. Run 'npm install' if needed, then run one build command.
 3. Launch one temporary local server with the lifecycle helper.
 4. Write 'baseline_eval_result.json' and stop.
- Do not perform open-ended exploration loops, repeated browser sessions, or speculative deep dives after every element already has a verdict.
- Do not invent elements that are not in the spec.
- The JSON must include at least:
 - 'game_id'
 - 'run_id'
 - 'final_verdict' ('pass' | 'fail')
 - 'confidence' (0-100)
 - 'checks_performed' (string list describing the checks you actually performed)
 - 'files_inspected' (string list)
 - 'commands_ran' (string list)
 - 'findings' (object list; each item must include at least 'severity', 'title', 'evidence')
 - 'limitations' (string list explaining what you could not verify and why)
 - 'summary'
- If some logic could not be verified because the entry path was blocked, the interaction path was unreachable, or evidence was insufficient, say that in 'limitations'. Do not pretend the check was completed.
- If you ran build steps, launched the page, used a Playwright smoke test, or performed static inspection, include them in both 'checks_performed' and 'commands_ran'.
- If you need a local server, do not run 'npm run dev' or 'npm run preview' in the foreground.
- Instead, you must use this exact lifecycle helper pattern:
 - Start:


```

          ""bash
          eval "$(("<LIFECYCLE_HELPER>" start --identifier "{game_id}" --timeout-seconds 30)"
          ""
          
```
 - Use '\$GAME_URL' for browser checks.
 - Stop before exit:


```

          ""bash
          "<LIFECYCLE_HELPER>" stop --state-file "$SERVER_STATE_FILE"
          
```

```

'''
- Do not finish while a dev server is still running.
- If you did not perform a check, do not fabricate it.
- Finish by printing one short terminal line that mentions 'baseline_eval_result.json'.

Parameters:
- 'game_id={game_id}'
- 'specification_path={specification_path}'
- 'run_id={run_id}'

```

D Review-and-refine Loop

Not every initially generated verification unit is immediately executable. After generation, each unit undergoes a review step in which the agent checks whether the unit is well-formed: whether the state s_k is constructible, whether the instruction i_k is unambiguous, and whether the expected outcome y_k admits a clear pass or fail determination. If any of these conditions are not met, the unit is refined iteratively until it yields a definitive verdict. This review-and-refine loop ensures that every verification unit entering the parallel execution stage produces a conclusive result, avoiding ambiguous outcomes that would undermine the reliability of the aggregate score.

E Data Synthesis Pipeline

To construct a benchmark for GameGen-Verifier, we build a dataset of game generation tasks, each consisting of an anonymized game identifier and a structured natural-language game specification. The pipeline proceeds in three stages.

Stage 1: Sourcing. We source candidate games from popular digital game marketplaces, specifically the Apple App Store [4] and Steam [39]. We collect a broad initial pool of games across seven target genres: Action, Adventure, Casual, Puzzle, Simulation, Strategy, and Board. We choose these genres because they are common marketplace-level game categories in the Apple App Store Games catalog and Steam’s genre/tag taxonomy, with Steam representing board games through Board Game/Tabletop tags [4, 40]. Games are initially filtered by popularity and quality, retaining only titles with substantial player engagement and high average ratings. We then apply rubrics-based LLM scoring.

Rubrics-based suitability scoring. Each candidate that passes the hard filters is scored by an LLM judge (GPT-5.4) across four rubrics on a 1–5 scale:

- **Textual Reconstructability:** whether the game’s core mechanics, rules, and progression can be fully specified in natural language such that the specification alone is sufficient for an LLM to produce a functionally correct implementation.
- **Rule Determinism:** whether the game’s rules are well-defined and unambiguous.
- **Mechanic Diversity:** whether the game exercises multiple distinct logic mechanisms (e.g., collision, scoring, state transitions, AI behavior, phase progression), providing sufficient variety for comprehensive evaluation.
- **Bounded Complexity:** whether the game’s scope falls within the generation capacity of current LLM-based coding agents, excluding titles whose scale (e.g., AAA-level content volume or engine complexity) makes faithful generation infeasible.

We retain games scoring at least 3 on every rubric and rank candidates by their aggregate score. The final curated set contains 100 games spanning all seven genres.

Stage 2: Specification synthesis. For each selected game, GPT-5.4 with deep search aggregates publicly available information and synthesizes a structured game specification. Each specification defines the core objective, controls, entities, rules, scoring, progression, failure conditions, and major mechanics. The specification abstracts away platform-specific assets and UI details while preserving the core gameplay logic, so that the specification alone is sufficient for an LLM to generate a functionally correct implementation.

Because the selected games are popular titles likely present in LLM training corpora, exposing the original game name during generation or evaluation could allow the model to rely on memorized knowledge rather than faithfully following the provided specification. To mitigate this data contamination risk, we anonymize all game identities: each game is assigned a unique identifier (e.g., Game-042), and all references to the original game name are removed from the specification. Both generation and evaluation operate exclusively on the anonymized identifier and specification.

Stage 3: Human refinement. Human annotators refine each specification for accuracy, completeness, consistency, and evaluability. This step removes ambiguity, normalizes terminology, and makes latent game logic explicit for both generation and evaluation.

Final dataset: VERIGAME. The final dataset contains 100 examples spanning seven genres. Each example contains an anonymized game identifier and a game specification.